Steffen Bollman & Bradly Alicea: “Open Strategies Interest Group“

Source: YouTube **Playlist**: 4th Applied Active Inference Symposium (2024)
Duration: 59:30 | **Uploaded**: Unknown **Transcript method**: auto_caption

---

hello welcome back we're here with real Bradley Lisa and recorded Stefan Bowman there will be a pre-recorded talk with some discussion too so thanks all and looking forward to the talk go for it okay see okay this coming through yes hey wonderful well then um yeah welcome everyone uh to my presentation about neurodes um my name is Jeff bman I'm a senior research fellow uh in at University of Queensland in Australia and sorry I can't be there live but the time zones are just too difficult um I want to start with why we build neurodes and then introduce what it is but we saw a couple of challenges and open signs and one of them is really non-reproducible workflows so we have um different great tools uh they run on lots of different data sets but the challenge is that if under Ling dependencies change they produce slightly different results and that's really not great if we want to infer anything on this data the other challeng that we saw is infrastructure disparities so lots of people have access to different Computer Resources so be their own notebook their HBC cluster or their cloud provider to process their data and wherever they want to run data analysis they need to come and install their software again and the software Wills be different across these installations and it often costs a lot of time again produces different results but also costs a lot of time in maintaining these different installations the other challenge that we see is data management so when you acquire experimental data then from an MRI scanner for example or behavioral data then we have to organize it some sense and that lots of different tools but we wanted to see if we can make that a lot easier and then also data storage and privacy so where we store this data how we collaborate with others how can we make this data openly available to others these are really questions that we had and thought well can we somehow make this nice and um nicer for people and easier to use so why is it so difficult um in the end all the software that they use for neuroimaging analysis are are software pieces that depend on other software pieces and these we often call libraries and what they do is these libraries provide functionality that you don't want to reimplement for example it's you have your software here that does some brain Imaging analysis and then you don't want to reinvent how windows are drawn on a computer or how data how data is plotted or how images are processed so you want to use what other people have already built to do these functionalities and then what happens is your software starts depending on these because you're using functionality from these and this usually works quite well and the reason is that we have in Linux for example we have package managers like app yam or if you use Python for your work you have cona and pip but pretty much all languages systems have some kind of package management system that keep the pendencies compatible with each other and that's really how it looks like it's like a puzzle piece where if one thing upgrades another thing has to upgrade as well they have to have matching versions and then uh we're hoping that this all works the challenge is with a lot of the scientific software that we work with it's often build by scientists who don't have lot of time they're not really software Engineers a lot of the time so what happens is they have unpackaged scientific software so these scientific software packages are not available in standard package systems and that means that if the operating system upgrades the package manager doesn't know about this software because it's un registered there and then what happens is the package manager changes something and because it didn't know about the software then it breaks and that's happens unfortunately quite often with scientific software and at leads to two problems uh again here's our reproducibility problem because if we just depend it on a library and even if it doesn't break let's say it just changes functionality our results will be different and that's really bad for reproducibility and also this means portability because if our software is tightly integrated with the system it means we can't install software on our notebook and then run exactly the same software on an high performance Computing system because that will have slightly different dependencies so we need to install it again often compile it from source code which can be a really lot of work so just to show this reil problem in a little bit more detail um this is a great article that looked at software that depends on gipc gipc is a very fundamental Linux library and what they did is they varied only that Library version um and they left the main piece of the software intact so they used free Surfer and lots of other pieces but I'll Focus here on free Surfer um so it's the same software installation just in different operating system just in different Linux versions in the end and what happened is in gilp C uh the exponential function for example was altered and was made more efficient but unfortunately this meant that the Precision of this function changed so now you you would think that this doesn't matter right it's like very uh a couple digits after um uh so yeah so it should be quite insignificant unfortunately it's not because free in particular has a it's a long running pipeline where we segment the human brain and this means that if you have small changes this accumulates over thousands of processing steps so this means in the end here they found differences in millimeter up to .2 millimeter in the cortex where um this is a change that's only based on a software version of this underlying GC so free was identical the only thing that changed was the operating system and the thing is this this was even statistically significant in some areas and that's really not what we want because if you acquire patient data on one software version and then you have matched controls on another software version then that would really lead to significant results that are actually meaningless because they're just based on computational noise so the organic Sol solution that I've seen in my career is that we just have people set up different computers and don't touch them don't try to update them because people are aware of these problems and this is not great because it's not a great use of resources and also it means we can't easily take our software and run it in different computers and this is this portability problem where let's say you want to run the same software on your notebook on your lab workstation in a secure environment on a cloud provider on a high performance Computing cluster or even on the instrument where you're acquiring your data like your MRIs you can't just easily uh build an algorithm on your notebook and then run it on the MRI scanner it's currently not possible and but it would be cool right for for translational studies in patients it would be fantastic if you had a model um that you build on your notebook you validate on lots of data and then uh you can actually deploy it in in real clinical environments that would be fantastic so this is really the problems that we're trying to solve with this problem with this project and we adopted one solution for this which is called software containers so I quickly want to introduce what software containers are and how they work because our software is based Bas on all of this so software containers if you've never heard of them a very simple thought pattern is um they're just boxes they're boxes to put software in and they're really like the stuff where you put food in and put it in your fridge it's fantastic because it keeps things separated and you can move them easily they're very portable and they don't depend on each other and that's really what software containers do they use specific Linux kernel features called namespaces and c groups they isolate processes and dependencies from each other so this means we can put everything our software needs into a standardized box and we can move that across operating systems and what it basically changes from our mental model when I said earlier we have this Lego piece where this this piece depends tightly on the other things in the system this now changes to where we take these dependencies we put them into the same box so now we actually don't have a dependency on the system anymore um the Box got a bit bigger because we have these dependencies now included but in the end stop is cheap but experiments scientific experiments are expensive so this is a very low price to pay for a lot of great features and the whole Cloud world everything the internet depends on today is based on software containers so this is really a very mature standard that has been adopted everywhere in the Computing industry so in science we're just we should just do it as well um so what we can do we can package all of our software in these boxes and then because we have these standardized interfaces now we can run this on Windows uh Linux and Mac on Linux this is a standard kernel feature so no performance penalties very easy to run on Mac and Linux we have some virtualization technology that we need but that got really good over the last years so you will not feel that it's slow it still feels very native but there is some virtualization there is some overhead but it's tiny compared to what we had a couple years ago so then let's come to what we built um before I show what we've built in eurodesk is I want to acknowledge all the people working on this project because yes I'm giving the talk here today but I'm just just one of many contributors um we started this project really during Co because we were all bored and sitting at home and we said well let's let's solve that problem why not right we have time um so we got together couple people all highlighted in in in bold here and said okay let's buil this we built a prototype during hackathon at the obbm and we got funding from the aidc to build a bigger version of this and then yeah we got lots of people involved um slide change sorry um we got lots of people involved from involved from across the world and lots more supporters like the big cloud providers AWS uh Google had supported us in the past as well um and then also the uh the university Cloud providers uh like egi Jetstream uh ADC nectar Cloud so that helps a lot and yeah we're currently funded by the mark Chan Zuckerberg initiative and welcome trust to build that uh system out further over the next years so this is what we buildt so I'll start here at the container level so this is really the software containers that I introduced earlier where we build a software container for every piece of software that people want in this so here we have a couple of Neuro Imaging pieces so FM free Surfer field trip ends and then we can control everything in there from the binaries the libraries that we have in there up to the actual software that people want in the end like m& which then depends on python which then depends on cona and so on so we can control all of these dependencies and then if you've used the software container before they are fantastic but they're not always easy to use like you have to run specific commands like Docker run or if you're on an HBC you need to run Singularity or obtainer run and we didn't want that people have to think about this and we build an accessibility layer we call it but in the end it's just an abstraction across multiple container technologies that we use and it's also a way how we can stream containers to end users so people don't have to download these containers so this this little block here was actually the most of the work that we spend on how do we get these containers to end users and how do we make it easy for people to use that so but ultimately what this allows us now to do is we can run these containers on any system so we can run this on Mac OS on Linux on the cloud on Windows on high performance Computing clusters in Jupiter notebooks like Google collab on the terminal pretty much anywhere and we try to indicate this here with this little script that goes between systems and that's not the thing that wasn't possible before we can code up an analysis on our notebook and we can move it across all these systems and will perform identically and behave identically across all of these systems so this really is fantastic for collaboration for transl of of results and um for reproducibility in general and then the community uses that interacts with the system but also finds problems bugs and then reports back and contributes new containers new things that people want to use in the system and that's really the ecosystem that we've built so the best way to get started with neurodes is go to our website which runs on the cloud or if you want to run a local version which runs on your computer and it depends on your use case if you have data patient data you definitely want to run this locally if you have open data you can use a hosted version and you're fine and then we have a more fine grain table down here where you can select what your compute platform is what your operating system is what the interface is that you use and then you get guided to the documentation where we guide you through how to run this on your system so in the end what you're getting is you're getting a virtual desktop which is just a Linux desktop with all the tools installed and the cool thing is all the software is ready to go so you don't need to install anything everything is there um we have no dependency issues so we can even have the same software in different versions so for example free server is almost impossible to install different versions of that software on your computer because it will replace dependencies that that other pieces depend on but because we we wrap all these dependencies in a software container we can actually provide all versions of neurodes of free Surfer for example going back over the last years without any problem because of this containerization the analysis reproducible uh down to the software level we're not reproducing anything Hardware effects I can show this later but yeah if there's like a CPU difference between Intel and AMD or arm we we don't control that but down to the software level it's reproducible and then this whole uh system is only 1.5 Gaby in size and the reason is we can stream all the software we need on demand in there so when you actually click on a software it is streamed live to this endpoint you can also choose to download it but then we realized we look at brain Imaging data and and check if things worked or didn't work then also we wanted to support high performance Computing systems so high performance Computing systems they don't often come with a desktop interface so you need to work from the command line so we build everything to integrate with these HPC systems so you can easily run neur desk on there often you can't install things on there so we took care of making that possible so you you don't need any elevated privileges if you have Singularity or obtainer already installed on your HPC and then we also support Jupiter uh and notebooks computational notebooks in general because we think this is a great way of documenting and anal analysis experimenting collaborating with other people and sharing results in the end so we really build it into Jupiter lab as well and provide with this uh a very nice interface to people that they already used to that I know how to work with then it's also very easy to contribute new software to it it's all automated on GitHub so we have GitHub actions where people can propose a recipe for a software this gets an automatically built tested and integrated into this whole system and made available and then we have a system which is called cvmfs this the streaming of software that I mentioned it's developed at CERN so CERN used cvmfs to stream software and instrument data across the world and we use exactly the same system and also collaborate with the people who run these servers so we have a primary server where we ingest all the software and then we distribute it worldwide so wherever you are in the world you should get a quick access to this software and um there's a client in there that localizes where you are and gives you the closest server and even if a server is down it just goes to the next server so it should be a quite Reliant system and we're using this since a couple of years and have never had any issues with that and then for the end user it just appears magically there they just see the software there and when they start it it just magically works it's just on the first start it's a little bit slower because it has to download certain things but then on the second start it's CED locally and it's it behaves like it was installed locally so the update in the community is pretty impressive and we saw really big growth over the last years so we have more than a thousand monthly users on our systems uh they come from over 60 countries so really worldwide adoption we see that more than 16,000 people downloaded our main software container that provides a lot of these things it's used in university courses for teaching it's used in conference workshops because it's fantastic for standardizing teaching material because you can't you don't have the problem that students have different software requirements on their computers they all log in into either a cloud-based system or they install our software container and then they can run all the same analysis which is fantastic and we also have various collaboration models that we support with this because uh we realized that in some instan it's nice to have a central compute instance where people all log in and they all work on the same data this is fantastic for uh open data sets this is fantastic if people are actually all on the same institution and they all collaborate on an Institutional server and that's really useful and used a lot also for teaching obviously that's a really great setup because teachers can then log into the other uh students noes and quickly have a look what's happening there if they struggle if they have a problem but then we also have this Federated model where every in let's say researchers are collaborating from different institutions they just install it on their infrastructure on their computer and they run analysis on their data and then they can for example aggregate high level results in a publication uh without sharing actually any confidential data so let's say brain extraction or brain uh let's say cortical thickness measurements right so you could easily share those without actually exposing the patient data that's underlying this and then then that can be shared across different institutions so that's a really cool model as well that we see often used with our our project and with this I claim that we made maintaining and installing scientific software fun again so it's now very easy uh basically you install neur desk on these different systems and we make sure that the software runs on these systems so even very recently we managed to get our containers running on an MRI scanner so this is a uh Works particularly well for the seens ecosystem so seens has a software platform called open Recon so we can actually build a software container that can directly be integrated into a reconstruction running on the system and um that is really closing the loop so now you could yeah acquire data build a CO model and then deploy it on an analiz scanner where patients can directly uh be scanned and the results end up in the pack system for people uh to look at so that's really uh great progress with the last months that we've made there so I want to quickly show the open data workflow in neurodes so this is something that we always focus on a lot we want to make consuming and producing open data a lot easier and what we've built there over the last years is that they support a lot of tools and databases that make that possible so I'll start with importing data so usually if you produce open data you would start from for example an MRI scanner or an EG system or you would download data from for example open Euro or the open science framework and then we have a tool in there that's called data that can handle a lot of these uh cases where we can download data from these systems also xnet we support which is a a tool commonly used in the Neuro Imaging space to Archive data and then we of course support any cloud provider or any local connections so pretty much anything this is not an not a complete list this is just a couple of examples what we support and then once the data is in the system we can process the data using pretty much any software out there if the data if the software is not yet there it can easily be added we currently have more than 100 pieces of software in there that that the community wanted and then we can organize the data and we can document our analysis so for this we highly work with the bit standard with the brain Imaging data structure but also bits has been extended to the EG world so we support this as well we support Jupiter as I said as a way of documenting this data but you can use also your own script system whatever you're used to you can install that um and then we wanted to make it very easy to share data with the community so we support Google collab for example so you can write your analysis in Jupiter and then you can provide for example a Google collab notebook with your paper where all the analysis run and we stream software into Google collab so with this we can even run software that wouldn't be able to run in Google collab so for example because you couldn't install it because Google collab doesn't have enough disk space for example to install a certain piece of software and we can actually run the software in Google collabs through our streaming service but also we made sure that we can easily upload data back to open Euro again through data lab or we can share data out to OSF or that we can share our analysis quickly on GitHub and then people can start again from this and can use our data that we produced or our our derivatives that we produced and then start the cycle again and then really yeah enrich the open ecosystem out there so people often ask what is our sustainability model uh will it be around in the next couple of years and of course that's a very good question because if you start adopting a framework like this you invest time in learning it and you want to be sure that it's around so we try our best there uh so we as I said we have been funded through an a platform Grant in the past we're currently funded by the mar and Chan Zuckerberg Foundation through welcome trust and we're applying for uh grants left and right to keep the system growing and developing but of course Grand funding is very competitive and of course Brands always want to fund things that are new they don't want to fund things in maintenance mode so that is definitely uh something that we anticipate so that's why we really want to build the community so we already have 10 more than 10 active contributors we have a lot of the infrastructure hosted through the community Through the open science grid uh the egi foundation where we don't have basically Cloud bills for us which keeps our costs very low which is good and also if we have help us from the community we can also lower our costs because we don't have to do certain work and then we provide neurodes as a service as well so this goes through a non-for-profit um Foundation which is called the Queensland cyber infrastructure structure foundation and what they do is they provide neurodes at a certain cost for for example a conference for a workshop or for University so let's say you don't want to use the open source software and do everything yourself you can go to qf and say I want this professionally managed I want a service contract I want an SLA I'm happy to pay money for that and I don't I want to use cloud resources for example and then cus can can provision that and set it up in a way that it's uh really at scale and working really well with with certain guarantees as well um and then this money flows back to us to develop things further and then we also have a lot of support from cloud providers so as I said we heavily run Cloud Technologies so we had been supported by Oracle cloud in the beginning then Google cloud and now currently our main sponsor there is AWS and the research clouds like egi Jetstream aadc which support us with Cloud resources and make us all possible so I want to quickly show you some results from this reproducibility aspect because I'm always claiming that neurodes is reproducible but is it and how reproducible is it so this is something that we looked at so PhD student in my group took that original paper that I showed earlier from Tristan clar where they found these reproducability issues with software so we took that paper and thought um tested can we reproduce the non-reproducibility of software so the analysis is the following uh we picked this part where we have data was scanned from an MRI scanner in anatomical space what they have to do is we have to register it to the m& space so this means we align it to a template space which FSL with which a with a tool called FSL flirt and then we take this data and we segment it so we segment subcortical areas and we uh measure for example the size of that so it's a very simple pipeline where you think that should be pretty easy to reproduce so what we did is we used a very similar setup than what clar used in their paper where we have a local installation of FSL so FSL is the same but then once we run on gipc versions 231 so on a Ubuntu 2004 at the neurodes we chose a quite old version of yuntu uh with the quite old version of GPC but the cool thing is the GPC version is consistent whereas here the GPC version is 2.28 so something in the middle it's on Alma Linux 8.5 and then also what we varied is weent varied the processor so here it's an an i7 and here it's an AMD appic processor so let's see what happens so we of course we compare these different systems now let's see what happens so the first thing is image intensity differences so the first thing is we were able to replicate the problems from the paper so if you have a local installation with two different gipc versions then you get different results and this is really intensity so we we only focus on the intensity in the area where we will later do an analysis so this actually the intensity change as whole brain but just to show where the problems are so we basically see that the the intensity of a boxel is like up to one one integer value different and this is small because when you think about the images are usually like 0 to 1,24 so a change of one doesn't matter much but still it's a change and it shouldn't be there so the cool thing is when we run this a neurodes where we control gipc the gipc version this effect is not there there's nothing there and that's exactly how it should look like there shouldn't be a difference between this it's a fully um deterministic analysis there shouldn't be any variance between that and that's cool so then when we look at the classification differences so this is now when we segment this data um we see again in the local installation with different GPC versions we see quite High disagreements between labels and this is now dice score so this is between zero and one and a016 dice score is quite a lot when you think about it um so this means we actually disagree in certain areas quite a lot between segmentation and again that shouldn't be the case now in neurodes you still see something there however note that the scale is an order of magnitude scale down so we had to scale it down to make anything visible there but we wanted to show it and why is that why is neurodes here showing some non-reproducibility because I just claimed that we are reproducible right and why are we suddenly not and we looked into this so we looked we traced the executions of this program um and we found that as expected in the local installation the software diverges so we're doing different things on different computers with different GPC versions and that's why we're getting different results so that explains it it's great on neurodes we're not diverging we're doing exactly the same thing on both computers so this means on a software level it's reproducible but why is there still a different result and the reason is that we use different processors was an Intel processor one was an epic AMD epic processor so different instruction sets different floating Point positions can still have an effect that we cannot control for even if we do everything the same we get still small differences they're tiny but they're still there so it's something to to note that 100% reproducibility will only be possible if you also control the hardware but then in the end you have to yeah have a trade-off there right when when is when is reproducibility enough uh and when how much effort do you want to put into having reproducible results there so yeah that's just uh our results there so a little bit of an Outlook what we're working on next so there's a lot of things we do work on but one thing that I really want to see live in the next years is interactive papers because that's what we can do now we can have an analysis in a jupyter notebook where we describe what we've done we can show all the results we can show all the analysis we could run everything and everyone could rerun it as well and then base their analysis on it and Fork papers and really say well my analysis based on that paper and have open data for pretty much everything and really uh yeah make it easy to to contribute to this open Science World and we already see some adoptions from this from from bigger journals like e life they already have like some of these executable papers so seeing this coming through there's still quite a bit of work required to make this easy and and very doable but uh our nees paper for example we already did that so the neur paper you can go and reproduce it and you can reproduce our reproducibility analysis for example which we thought was quite nice so with this I'm at the end uh of my talk so I just want to highlight that um it's an open community so if you uh we we need we need Community we need Community engagement so if you like what we've done if it's useful for you get in touch help us build new things find bugs and um yeah we also have a little bit of time still left to show neurodes connection if this is something you want to see go to our website um neurod des. org click on hosted and then you see we have different services and I'll said we choose the place service with different regions Australia us and Europe so if you're roughly in one of these regions pick that one so I said I'll pick Australia I just want to uh quickly highlight here that all of these servers require some kind of a login Australia you need to be an Australian researcher because that's how the service is funded in the US and Europe you need to have a GitHub login at least um so I'll click Australia then what happens is we'll authenticate you with an account in case of Australia we use an Australian access account in then you just say Okay I want a default environment there shouldn't be anything else available for you this will start up uh a Jupiter Notebook on cloud resources um that are in that country so you don't want to put patient data there of course but you can put uh open data there and in terms of storage uh you have a home directory which has about 2 gigaby of storage you will get a warning if it's full and then nothing will work anymore so you need to be care that this is not happening and then everything else you change on the system will be reset after after your session is done but that's basically how it looks like you go into this system and then it's just a very familiar Jupiter notebook system so I quickly want to show now um we have for example an example repository which I quickly want to show so I'll just I'll just show it on the website first so what we've done is we build an example repository of lots of different workflows where uh we for example show how to use pipeline systems on here we show how to do diffusion analysis for MRI uh lots of lots of different things and we're currently building that out so if you have a tool that you want in there please go and contribute an example contribute a tool and what we can also do is I can I can easily take this and also run this on our website so for this uh we will just just check out this repository so I'll just show that um so just clone our example notebooks and then that will take a couple yep so here our example notebooks and then we can just pick an example here so I'll pick structural Imaging and then I'll take the uh I'll take the functional Imaging where is it intro to pre-processing I just wanted to have a very quick one to show yeah okay so night pipe short so that's a really quick one to show so um what we do is we have a first cell where we set up neurodes this is not needed if you run this in a neurodes environment but it's needed outside for example in Google collab and then we output which CPU are running on Just for reputability reasons and then you can import any software into this notebook so for example here I install I import FSL and this is a software that we provide so this software will now get string into this notebook and then we can run any tool from the software you can also install any python libraries anything um you need but yeah that was uh really the the streaming that you saw so that this B tool was now streamed into this notebook and now can be used in anything and this the same way we can load pretty much any software and have it also documented what software we're using in which version we're using it and then I'll just uh download some test data and and show that so I'll quickly run this note book um while this is running I want to show something else because notebooks are great but sometimes it just needs more it needs a desktop and this is what we can do so we can launch a full virtual desktop in the system which should look familiar if you've used the Linux desktop before but ultimately uh it has a little start button down here where you go into NE desk menu and you see we have lots of different categories right now for functional Imaging electrophysiology diffusion Imaging and then you can start pretty much anything from here with visualization software and you can just start the software it should start it should run and should behave exactly as what you're used to as I said the first startup will always be a little bit slower um because it has to stream the software but then you you can browse brain Imaging data and access everything you want so now I want to show quickly as well um one more one more way of of using that so I said earlier we have this this jup notebook so I can also open this in Google collab so when I open this in Google collab I'll just do the same thing here run all this is now really cool because this is what I mentioned like this you could attach to your paper where you have your whole analysis coded up in there and then run it on on Google collab and it will be behave identically than what you just built on the neur desk instance for example um while this is running um see yeah I'll keep this running I just want to show one more way of doing it so uh oh yeah okay but that will not work currently just sharing the screen because I only share the browser but yes so then there's ways of how you can run it locally on your computer uh with a Docker container so you only need Docker installed it can run on high performance Computing clusters pretty much anywhere where you should need it it should run um and when I just go back to our website quickly um yeah your high performance Computing system we have instructions here how to do this even if you don't want this whole setup around we have ways of just installing a single container and you can use that in Docker or in Singularity so it's really a a very layered approach of this whole system so if you just want everything and no work then just take the top hosted versions and then you can just go down the technology stack and use what you need for your work that's really uh what we've built and let's see if that now finished by now so this this will still take some time um let's see I'll see if this one is finished by now um yeah so basically now this Rand analysis it downloaded data and then maybe I can show that one see uh we also have built in visualizations which is quite nice so we can in a jupter notebook this is using a library called miew where we can browse uh and visualize neuroimaging data so yeah so the notebook is still running so that's why it's not executing it yet but yeah once it's once it's getting there it will um so yeah that's really that's really what we've built so yeah um while this running we can we can take some questions uh Bradley if you have anything that people might want to ask um I can show that well wait but otherwise that's pretty much what I had prepared yeah that sounds good thank you um I'm a little I'm interested in the containerization aspect of this so this is you know something that you can run kind of on any machine on any kind of computing um Paradigm like you know if you want to run on the cloud or on your laptop corre so a lot of the people in the group you know they're interested in active inference they're interested in sort of building their own tools and then how maybe to do containerization as part of that strategy yes so can you speak to that a little bit yeah exactly exactly so if you build your own tools um you can just basically build them in here just get them to work first and then once they work I'll quickly show how this would look like then um when you want to containerize it um so for example let's say it's a it's a python tool or a mlab tool mlab we have to compile and then we download it I'll just uh I can show that quickly so for example SPM as an example there so SPM is a brain uh brain imaging software as well and if you're ready to containerize it you can send us this build.sh file where you basically build a container uh based on a certain Ubuntu version and you set everything up yourself and here's an example where we download a compiled zip file for mlab that then runs in a mlab runtime environment and we have lots of tricks to make this recipe quickly quickly writable so this is actually a higher level language which is called which is developed by neuro Docker team and this higher level language makes easy to build containers because if you if you want to build containers yourself there's actually a lot of optimization that has to go into it and with this higher level abstraction makes it easier to to build these containers but then yeah once you have a software it's pretty easy to run it in here and even if you don't want to containerize it if you got it to work in here and you have the steps to do it it will run on any neurodes system pretty much so that's the uh that's the cool thing so if you have a python library and it's just three steps to install it you can do that as well you don't have to containerize it only if you really want to freeze it in time make it really um robust across different systems then this is the way to go yeah so does this make sense yes yes and then I I would like to ask you know what does the future look like I mean do you have a road map where you're going out five years or so and and if so what are the needs that you need to kind of address in that road map yeah exactly so yes we have lots of ideas how to how to go forward with this it always will depend on funding what we can do in terms of new features one thing we do want is and we're heavily working on this right now is getting into the clinical space more to make it very easy to run the software anywhere so currently we're still need needing some containerization like Docker podman Singularity obtainer we want to and we we just have a prototype for that where we actually bring everything we need like we use a quo based emulation or virtualization and if the virtualization framework is enabled on a computer it's not even emulated so it's really native speed and with this we can run anywhere on any system really and we have no limitations anymore currently you still need some privileges to start a container on a system and this is really needed for these clinical um scenarios where we want to get into so this is one thing we're building out running these containers on an MRI scanner that will be a lot of work in the next months and then uh the other thing is lots of other communities have contacted us so this is now focused on brain Imaging right but active inference might have lots of different tools there and they might want to build their own flavor of this like an active inference neurodes and we have communities like the genomics community microscopy uh Dermatology they want to do this as well so what we will be doing is we will refactor everything that's not neuroimaging and put that into a new project and then we have spin-off projects like neurodes that use this underlying technology but then they build the software around it and then Community maintains that one um and builds their own flavor of this this is something we're going to do in the next years um we still need some funding to accomplish that but I'm thinking that is yeah that is absolutely needed to support other communities there and there's also a whole underlying thing of what cloud technology we're using right so all of these things don't have to be reinvented if another Community wants to use that so that's that's really our road map for the next years making that yeah more General and yeah and specific at same time like more clinical as well yeah well thank you yeah it's good cool okay um yeah not sure so I know some visitations came through so I can quickly show that so this is something which is I think really valuable for a lot of brain Imaging people so you can just quickly yeah look at the brain which is really cool in in Ain browser so you don't need any fancy software there and yeah that's what for example in this case new makes possible and I think this is really our our fundamental um philosophy with this right we want to just bring the right tools to the right people and make it easy to use them so they can do awesome work that's really what our project is about yeah thank you very much and wonderful people if people have questions after this talk shoot me an email um and contact us through GitHub or gith have discussion forums and we're happy to help thank you right thank you yeah all right that's all we have for that um I don't know if we have any questions we want to ask I can try to answer them okay if people have any questions in the live chat they can add but thanks for sharing this and for bringing this important flavor into the mix um where it took me was right where you asked at the end with stripping back to the desk and then allowing the spin-offs and thinking about how we could use that and just how it's like the through line of the education the research the application that's kind of tracing back the community and information supply chain to Applications of activ inference so when we come to a symposium like this or think about planning a symposium like this think about making a startup or something like that using this in an application setting and we trace it back and then that's where these topics like reproducibility and research Integrity are the foundations of the application and if there's ambiguity in all these different threads and loose ends like from the availability of the papers things that have been focused on in science across fields and then these really specific computational issues and then the example with a versioning of a library having these cascading consequences to say nothing of like security issues it's just completely it's like an empirical open science point that was very well made and so now where do we go from that observation right right yeah I mean i' I've had experience with containerization um you know in the openworm foundation which I also work with uh they they've developed a container for demos and they run their simulations in a series of demos and the idea is to be able to get people to use the demo and get everything to run you don't need any dependencies on your machine you just worry about running the the container and that's it and it's been successful I mean it's a little tricky to use and get to work but you know the idea is to make things reproducible so if you see something in a paper or see something that I show you a demo of you can reproduce it for yourself and it's everything's the same so you get the same answer and you can even maybe like modify some of the variables or parameters in that model and see you know work with it on your own and know that that's uh you know that what you're seeing is actual an actual result not just some noise or difference that comes from the environment yeah um like practically what are some interesting ways that as we digest all of this year and think about what projects like as a scientific Advisory board or just as an Institute or as part of coalitions what would be interesting how how do you how do you see that popping up across these different research communities yeah I think well for one you could use some of the packages that we've seen in the Symposium you could run them in a container and it would help people if you're running them in different locations like if you're in a research group or if you're in a uh just a casual you know learner who's trying to run an analysis it would equalize the ability to run those tools but also you know we can develop demos um if people are interested where you show you demonstrate active inference and you put it in a container so someone might run a contain you know someone who is learning about active inference could run a container and they could run these simulations and they'd be you know able to run them no matter where they are and it doesn't take them a lot of effort to sort of install dependencies cuz sometimes and when you want to say demonstrate something in a simulation you need to have people run like you know a bunch of things on top of one another and not everyone can either do that or can run them on their own machine successfully so that hinders what we can do in terms of demonstration and Education and Research so that just takes care of that layer and then everything else is like you can worry about like doing the good stuff doing the demos doing the science as you've had many years of working in open source like what heris discs or intuitions do we have in our toolkit when again just cuz something could be open source doesn't mean it necessarily should so how do we add signal not noise or Worse how do we navigate between getting overwhelmed with the technical complexity of trying to take on all of these new research practices and and just like we we could spend all all of our computer time making something reproducible I I I I I don't know exactly where I'm going with that but just how do we guide and just put out especially if we're putting out capacities related to cognitive modeling perception control then is there any kinds of considerations that we should think about before just releasing really effective containers for that yeah so I mean I guess what's in a container is basically the software that you release as open source and of course there has to be a strategy for releasing open source software you just don't dump software out on the web or you just don't like kind of produce a bunch of files and then you know with limited documentation until people have at it every group has their own strategy and a lot of that is you know to think about the utility of what you're releasing as an open source package so sometimes you're interested in the uh commercial value sometimes they proprietary value in releasing something or not sometimes you release you have code but it's not well documented so the utility there is limited and that people can't reuse it so that doesn't necessarily make it good release um you know so you want to be able to have I think and I I say this as being a full supporter of Open Source activity but to release things that are very Polished in the sense that you know why you want to release it and you have the right support in place and sometimes groups will have the resources to do that and sometimes they won't so if you're a small academic lab then you don't really you can't spend a lot of time you know polishing the software uh in the talk I I gave early I think it was yesterday I talked about like kind of these practices of software engineering and that sort of thing and that's something that not every group can do so that means that we maybe need either organizations to kind of have that layer of you know uh making things work well before release them or having meta organizations that can help scientific organizations do these kind of releases and I'm against like making something open source for like a paper I'm just saying that like in tool building you know you have to have a deliberate strategy for it and it's it it pays off if you do yeah that's very wise like your talk in part one and seeing this again and um the prior talk on NGC learn and the Museum of examples and going Beyond just curating links to to hosting more and more and that is what will allow Learners to have access to these tools build their intuition about the topic maybe continue on and kind of um converge into the development attractor but even finding friction points and bugs and Reporting it and just putting up a yellow flag or just asking it the question a lot of times it feels like people who are outside of the development don't realize that those visibilities that and experiences that they have into the outcome can be in the blind spots or just forgotten by by bandwidth constrained developers and then that is like our research Community with people who are at different stages and using tools differently so it's like how could we not have this yet it's hard to to hold that yeah and it is it it is a Commons for for science which wasn't really part of the pre- internet pre-computer research world and so there's all these forms and functions that we barely barely even able to describe let alone what has really been tried in different epistemic niches so it's it's super exciting because there's so much experimentation across multiple scales people doing things on their own being their own program manager on through just with friends working with AI and the kinds of things people are asking and the artifacts they're getting to and then even up to like the metago type coordinations of organizations who want have Commons and be particular but also find the interfaces that will help them do things together they can't do separately yeah um do you have any sort of last thoughts or question questions and what would be some exciting directions for you and or The Institute in the year to come yeah so I mean uh Daniel asked me to contribute something on this topic and and kind of in the abstract because we do it looks like we do a lot of open source and and things and it's just kind of you know to develop a strategy there to develop a vision there as the thing that we want to get to so um you know I would say that like um you know one thing might be and I think this is very useful is to have a sort of educational component where you can have some you know tools for active inference and to teach people sort of how to you you know we we usually use simulations as an educational tool and making simulations are you know a nice tool because you people can interact with them and so you know and it doesn't have to be like a very highle simulation be a very simple thing that demonstrations of things demonstrations of Concepts where we might package uh some sample data with a sample simulation and say here build your own sort of active inference model and you know kind of walk people through how to do it and then eventually they'll start playing with it and they'll say oh this is great and I want to get in more into some of these topics and then that's that's sort of a Gateway into the institute for people who may not feel comfortable with a lot of you know kind of diving right into the technical jargon they want to have like a a nice sort of way to ease in and and something that's intuitive and then that helps them um but other things you know like like as Stefan said you know that kind of the thing that they're working on neurodes is for Neuro Imaging but you can use different versions of this for you know different sorts of things and there're also different versions of Linux that you can build like your own custom version of so uh in the talk that I gave earlier there's uh neurod Debian which has the ability to build these builds where you build them as like a specialized operating system for a certain thing they have neurod Debian which is for Neuroscience but there can be other versions of Debian for like you know musicians or data scientists and maybe you know I don't know if it's useful but but to have something for active inference with all the tools included and you just you know download that that re that drro and you get the tools and they they work thank you these are all great directions a closing note I think it'd be very appropriate in a history Rhymes kind of way that the more generalized cybernetics arose out of neuroimaging and we saw SPM package in one of the presentations and then also neuroimaging and the special constraints like computational resources patient privacy those constraints in that community of neuroimaging also leading the way on the tooling which we can also learn from with more General systems yeah yeah okay thank you for this all right see you thank you